

第26回 実践：Web ゲームプログラミング5（当たり判定）

実習時のフォルダ構成

Web_名前 の中に 26 フォルダを作る。第 25 回の `sample` フォルダと `exercise` フォルダを、26 の中にコピーして、続きから作る。

```
Web_名前
└── 26
    ├── sample          ← 第 25 回の sample をコピーする
    │   ├── index.html
    │   └── main.js
    └── exercise        ← 第 25 回の exercise をコピーする
        ├── index.html
        └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～3、発展）

講義内容

`sample/main.js` に書き足す。書いたら `sample/index.html` をブラウザで開いて確認する。

1. 前回の復習

- 敵は、`frameCount % 60 === 0` のとき（約 1 秒に 1 回）、乱数で決めた x 座標に出てくる。
- 弾も敵も、「`push` で追加 → `for` で動かす → 画面外に出たら `splice` で消す」の流れで処理している。

今回は、弾が敵に当たったら、弾と敵を消して、得点を増やす。

2. 今回の `main.js` の並び

```
// ===== 準備 =====
// ===== ゲームの状態 ===== ← 得点の変数を追加する
// ===== 自機・弾・敵 =====
// ===== キー入力 =====
// ===== 更新 =====          ← isHit と checkBulletHits を追加し、update から呼び出す
// ===== 描画 =====
// ===== ゲームループ =====
```

3. 得点の変数

「ゲームの状態」の部分に、`score` を追加する。

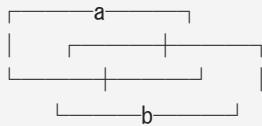
```
// ===== ゲームの状態 =====
let score = 0;
let frameCount = 0; // ゲーム開始からのフレーム数
```

4. 四角形の当たり判定 (isHit)

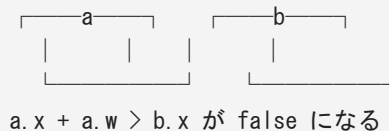
2つの四角形 a と b が重なっているかは、次の4つの条件で調べられる。4つが**すべて true** のとき、重なっている。

条件	意味
$a.x < b.x + b.w$	a の左端が、b の右端より左にある
$a.x + a.w > b.x$	a の右端が、b の左端より右にある
$a.y < b.y + b.h$	a の上端が、b の下端より上にある
$a.y + a.h > b.y$	a の下端が、b の上端より下にある

重なっている例



重なっていない例 (a が b より左にある)



- 1つでも **false** になれば、横か縦のどちらかで離れているので、重なっていない。

「更新」の部分の `updateEnemies` の下に、`isHit` を追加する。

```
// 2つの四角形が重なっていたら true を返す
function isHit(a, b) {
  return a.x < b.x + b.w && a.x + a.w > b.x && a.y < b.y + b.h && a.y + a.h > b.y;
}
```

- 第20回で学んだ「真偽値を返す関数」。引数 **a** と **b** に、**x・y・w・h** を持つオブジェクトを渡す。
- 弾も敵も自機も、同じ **{ x, y, w, h }** の形なので、どの組み合わせにも同じ `isHit` が使える。

5. for の中の for (入れ子) と break

すべての弾と、すべての敵の組み合わせを調べるには、`for` の中に `for` を書く (`for` の入れ子)。

「更新」の部分の `isHit` の下に、`checkBulletHits` を追加する。

```
// 弾と敵の当たり判定
function checkBulletHits() {
  for (let i = bullets.length - 1; i >= 0; i--) {
    for (let j = enemies.length - 1; j >= 0; j--) {
      if (isHit(bullets[i], enemies[j])) {
        bullets.splice(i, 1);
        enemies.splice(j, 1);
        score += 100;
        break; // この弾は消えたので、次の弾へ
      }
    }
  }
}
```

(1) for の入れ子

- 外側の `for (i)` で弾を 1 つ選び、内側の `for (j)` で、その弾とすべての敵を比べる。
- 内側の `for` が最後まで終わると、外側の `i` が 1 つ進み、次の弾で、また全部の敵と比べる。
- 内側と外側で、別の変数名 (`i` と `j`) を使う。同じ名前にすると、正しく数えられない。

弾 ■	比べる敵 ■
弾 2	敵 1、敵 0
弾 1	敵 1、敵 0
弾 0	敵 1、敵 0

(弾 3 発・敵 2 体のとき。どちらも削除があるので、後ろから調べる。)

(2) 当たったとき

- 弾と敵の両方を `splice` で消し、得点を 100 増やす。

(3) break

- `break;`：その `for` 文を、途中で終わらせる。`for` が入れ子のときは、`break` を書いた内側の `for` だけが終わる。
- 弾 `i` は、当たった時点で消えている。そのまま内側の `for` を続けると、消えた弾 (`bullets[i]`) を調べようとして、エラーになることがある。
- そこで `break` で内側の `for` を終わらせ、外側の `for` で次の弾へ進む。

6. update から呼び出す

「更新」の部分の `update` を、次のようにする。

```
function update() {
  frameCount += 1;
  updatePlayer();
  updateBullets();
  updateEnemies();
  checkBulletHits();
}
```

- 当たり判定は、弾と敵を動かしたあとに行う。
- 弾を当てると、弾と敵が消える。

7. 得点をコンソールで確認する

得点を画面に表示するのは、次回。今回は、コンソールで確認する。

- F12 の開発者ツールのコンソールに、`score` と入力して `Enter` キーを押すと、その時点の `score` の値が表示される。
- 敵を倒したあとで、もう一度 `score` と入力して、値が増えていることを確認する。
- このように、コンソールに変数名を入力すると、ゲームを動かしながら、変数の値を確かめられる。

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 当たり判定を作る

- 講義と同じように、`score`・`isHit`・`checkBulletHits` を追加して、弾が敵に当たると、弾と敵が消えるようにする。
- 1 体倒したときの得点は、自分で決める。
- コンソールに `score` と入力して、得点が増えていることを確認する。

演習 2 `isHit` をコンソールで試す

- コンソールに次を入力して、`true` になることを確認する（2 つの四角形が重なっている）。
- `isHit({ x: 0, y: 0, w: 10, h: 10 }, { x: 5, y: 5, w: 10, h: 10 })`
- 2 つ目の四角形の `x` を `20` に変えて入力し、`false` になることを確認する。なぜ `false` になるかを、4 つの条件のどれが `false` になったかで説明する。

演習 3 `break` を確かめる

- `checkBulletHits` の `break;` の行の先頭に `//` を付けて、たくさん弾を撃ってみる。コンソールにエラーが出ることを確かめる。
- 確かめたら、`//` を消して元に戻す。

発展（任意）

- 敵に HP（2）を持たせ、弾が 2 発当たると倒れるようにする。1 発当たったときは、弾だけを消して、敵の HP を 1 減らす。
- HP が 1 になった敵は、色を変えて描く。